

## Lab: Filesystem Access

### Resource vs Tool for Files

This is a real design decision you will face when building MCP servers.

#### Resources

- Resources are passive, read-only data sources for context.
- Each has a URI and MIME type.
- Discovery uses resources/list (direct) or resources/templates/list (parameterized)
- Retrieval uses resources/read.

#### Tools

- Tools are model-controlled actions.
- The official MCP reference filesystem server (@modelcontextprotocol/server-filesystem) exposes reading as tools (read\_text\_file, list\_directory, ... all marked readOnlyHint: true), restricted to the folders you pass on its command line.

#### Why Tools Instead of Pure Resources for Files?

- A resource template like docs://{name} matches a single URI segment — it cannot cleanly address an arbitrary nested file tree.
- The model often needs to actively decide which file to read mid-task — that is tool (model-controlled) behaviour, not passive context.

*This lab uses both deliberately: a docs://{name} resource template for top-level documents, plus read\_file / list\_documents tools that handle nested paths the way real filesystem servers do.*

**Important:** the server decides what it exposes. It never trusts a path from the model or the client — every path is resolved and checked against its own ROOT at the operating-system level.

### Build a Sandboxed Filesystem Server

#### Step 1 — Set Up the Project and Test Directories and Files

Create the project and sample documents:

```
mkdir mcp-fs-resource
cd mcp-fs-resource
pip install "mcp[cli]>=2.0.0"
```

Create the test folders and files:

```
# macOS / Linux
mkdir -p documents/notes not-in-documents
echo "This is content of ReadMe.txt" > documents/readme.txt
echo "This is content of TODO" > documents/notes/todo.txt
echo "This is content of TODO (outside)" > not-in-documents/todo.txt

# Windows PowerShell
New-Item -ItemType Directory -Force documents\notes, not-in-documents | Out-Null
Set-Content documents\readme.txt "This is content of ReadMe.txt"
Set-Content documents\notes\todo.txt "This is content of TODO"
Set-Content not-in-documents\todo.txt "This is content of TODO (outside)"
```

## Step 2 — Create fs\_server.py

```
"""Filesystem Resource Server - read-only access to documents under a sandboxed root.

Security model:
* Sandbox: only files under ROOT are ever exposed.
* Path containment: every request is resolved and checked to stay inside ROOT.
* Read-only: no write/delete/move operations are exposed.
"""

import os
import sys
from pathlib import Path

from pydantic import BaseModel

from mcp.server import MCPServer
from mcp.server.mcpserver.exceptions import ResourceError, ResourceNotFoundError, ToolError
from mcp.types import ToolAnnotations

# Sandbox root: only files under here are exposed. Override with DOCS_ROOT.
ROOT = Path(os.environ.get("DOCS_ROOT", "./documents")).resolve()

mcp = MCPServer("Filesystem Resource Server", instructions=f"Read-only documents under {ROOT}.")
```

```
class DocumentList(BaseModel):
    root: str
    total: int
    documents: list[str]

READ_ONLY = ToolAnnotations(read_only_hint=True, destructive_hint=False, open_world_hint=False)

def _safe_path(relative: str, error: type[Exception] = ToolError) -> Path:
    """Return the absolute path for `relative`, but only if it stays inside ROOT."""
    absolute_path = (ROOT / relative).resolve() # resolve() collapses .. and follows symlinks
    if not absolute_path.is_relative_to(ROOT):
        raise error(f"Access denied: '{relative}' is outside the sandbox")
    return absolute_path

@mcp.tool(annotations=READ_ONLY)
def list_documents(subdir: str = "") -> DocumentList:
    """List readable documents under the sandbox root (recursively), as paths relative to the root."""
    base = _safe_path(subdir)
    if not base.is_dir():
        raise ToolError(f"Not a directory: '{subdir}'. Call list_documents with no arguments to see the tree.")
    files = sorted(p.relative_to(ROOT).as_posix() for p in base.rglob("*") if p.is_file())
    return DocumentList(root=str(ROOT), total=len(files), documents=files)

@mcp.tool(annotations=READ_ONLY)
def read_file(filepath: str) -> str:
    """Read any document (including nested ones) by its path relative to the sandbox root, e.g. notes/todo.txt."""
    target = _safe_path(filepath)
    if not target.is_file():
        raise ToolError(f"Not a file: '{filepath}'. Use list_documents to find valid paths.")
```

```

return target.read_text(encoding="utf-8", errors="replace")

@mcp.resource("docs://{filename}", mime_type="text/plain")
def read_document(filename: str) -> str:
    """Read a top-level document as an addressable resource (read-only context)."""
    target = _safe_path(filename, ResourceError)
    if not target.is_file():
        raise ResourceNotFoundError(f"No document named '{filename}'")
    return target.read_text(encoding="utf-8", errors="replace")

if __name__ == "__main__":
    print(f"Serving read-only documents from {ROOT}", file=sys.stderr) # stderr, never stdout
    mcp.run()

```

The server exposes two tools and one resource template, with every path funnelled through a single security chokepoint (`_safe_path`):

- `list_documents` — lists all files under the sandbox root (recursively) and returns structured data: root, total and documents
- `read_file` — reads any file (including nested ones) by its path relative to the sandbox root, e.g. `notes/todo.txt`
- `read_document` (resource template `docs://{filename}`) — reads a top-level file as an addressable resource; a missing file returns protocol error -32602
- `_safe_path` — the path-containment guard: `resolve()` collapses `..` and follows symlinks, `is_relative_to(ROOT)` rejects anything outside
- `ToolError` / `ResourceError` — carry a clear message back to the client; any other exception would be masked as "Error executing tool ..."
- `readOnlyHint` annotations — tell the host these tools never modify anything, so it can show a lighter approval prompt

### Step 3 — Test with MCP Client

#### `fs_client.py`

```

import asyncio
import sys

```

```
from mcp import Client, MCPError, StdioServerParameters

server = StdioServerParameters(command=sys.executable, args=["fs_server.py"])

async def main():
    async with Client(server) as client:
        # Discover what the server offers
        templates = await client.list_resource_templates()
        print("Templates:", [t.uri_template for t in templates.resource_templates])

        # Resource access - inside the sandbox, should succeed
        readme = await client.read_resource("docs://readme.txt")
        print("Resource:", readme.contents[0].text)

        # Tool access - inside the sandbox, should succeed
        listing = await client.call_tool("list_documents", {})
        print("List:", listing.structured_content)
        listing = await client.call_tool("list_documents", {"subdir": "notes"})
        print("List notes:", listing.structured_content["documents"])
        content = await client.call_tool("read_file", {"filepath": "notes/todo.txt"})
        print("Read:", content.content[0].text)

        # Path containment - attempts to escape the sandbox, should fail
        for args in ({'filepath': '../not-in-documents/todo.txt'}, {'filepath': '/etc/passwd'}):
            result = await client.call_tool("read_file", args)
            print("Blocked:", result.is_error, "-", result.content[0].text)
        result = await client.call_tool("list_documents", {"subdir": "not-in-documents"})
        print("Blocked:", result.is_error, "-", result.content[0].text)

        # A missing resource is a protocol error (-32602), not a tool result
        try:
            await client.read_resource("docs://missing.txt")
        except MCPError as exc:
```

```
print("Missing resource:", exc.error.code, exc.error.message)
```

```
asyncio.run(main())
```

Run it from the mcp-fs-resource folder (the client starts the server):

```
python fs_client.py
```

Expected output:

```
Templates: ['docs://{filename}']
```

```
Resource: This is content of ReadMe.txt
```

```
List: {'root': '/path/to/mcp-fs-resource/documents', 'total': 2, 'documents': ['notes/todo.txt', 'readme.txt']}
```

```
List notes: ['notes/todo.txt']
```

```
Read: This is content of TODO
```

```
Blocked: True - Error executing tool read_file: Access denied: '../not-in-documents/todo.txt' is outside the sandbox
```

```
Blocked: True - Error executing tool read_file: Access denied: '/etc/passwd' is outside the sandbox
```

```
Blocked: True - Error executing tool list_documents: Not a directory: 'not-in-documents'. Call list_documents with  
no arguments to see the tree.
```

```
Missing resource: -32602 No document named 'missing.txt'
```

Done when: the two in-sandbox reads succeed, all three escape attempts come back with `is_error` `True`, and the missing resource returns `-32602`. Try it with a symbolic link inside documents that points outside — it is blocked too, because `resolve()` follows the link before the check.